MyCodeSchool

teorie — contract, upsell, pipeline

fluxul de business · datele din 3 tabele · rând → obiect · așteptarea în pipeline · exercițiile 05 și 06 · Andreea

1. Fluxul din fișa ta

```
CONTRACT (ce s-a vândut)                ENT (ce primește clientul)
─────────────────────────               ──────────────────────────────────────

1. Book contract                ──────► Tenant 1000   Provided
   01.01 – 31.12.2026                   Phase  1001   Open          01.01 – 31.12
                                        Request 5001  In Progress

4. Book upsell                  ──────► Phase  1001   Completed     01.01 – 29.10
   pe 30.10.2026                        Phase  1002   Open          30.10 – 31.12
                                        Request 5002  In Progress   (tenant: neschimbat)

7. jobul din fundal             ──────► 8. SCHEDULE ACTIONS: toate pe Done
```

2. Cele 3 tabele și ce le leagă

Tabel	Ce ține	Cauți după
`/CFF/CONTRACTS`	documentele: booking, upsell	`CONTRACT_ID`
`/CFF/ENT_ITEMS`	tenant, faze, request-uri	`ENT_ID`
`/CFF/SCHED_ACTIONS`	acțiunile programate	`DOC_NO`

```
CONTRACTS ──ENT_ID──►  ENT_ITEMS
     │
     └──────DOC_NO──►  SCHED_ACTIONS
```

Regulă: ID-ul îl **citești** din primul tabel și îl folosești în al doilea. Nu-l scrii de mână în test.

3. Ent, înainte și după upsell

după booking

Tenant	1000	Provided	01.01 – 31.12
Phase	1001	Open	01.01 – 31.12

după upsell pe 30.10

Tenant	1000	Provided	01.01 – 31.12
Phase	1001	**Completed**	01.01 – **29.10**
Phase	1002	**Open**	**30.10** – 31.12

Regula dintre rânduri	Cum o verifici
faza nouă începe **a doua zi** după ce se termină cea veche	`findPhaseGaps(items)` → `[]`
o singură fază e `Open`	`filter(...)` → `toHaveLength(1)`
tenant-ul nu se schimbă la upsell	același rând în `expected`, la ambele cazuri

4. Rând → obiect → test

```
celulele din SAP          pagina (pages/)              testul (tests/)
─────────────────         ──────────────────           ──────────────────────────────

DOC_NO | DOC_TYPE | …  ─► readRow(0)  ─► { docNo, docType, … }
                          readRows()  ─► [ {…}, {…} ]  ─► expect(…).toEqual(c.expected.documents)
```

Folder	Ce are voie	expect?
`models/`	forma unui rând (`interface`)	nu
`pages/`	caută și citește; aici stau numele coloanelor	nu
`utils/`	cazurile, regula fazelor, fișierul cu ID-uri	nu
`tests/`	compară ce a citit cu ce se așteaptă	da

Ordinea într-un test: caută → așteaptă numărul de rânduri (`resultCount`) → abia apoi citește rândurile.

5. Defect cunoscut: `test.fail`

Datele din SAP	Testul marcat cu `test.fail`	Ce faci
încă greșite (30.12)	verde	nimic, defectul e notat în caz
reparate (30.10)	roșu: „Expected to fail, but passed”	șterge `knownDefect` din caz

6. „Wait for 2 hours” nu stă în test

	Cum	De ce
nu	`waitForTimeout` de 2 ore	ține pipeline-ul ocupat; dacă pică, afli după 2 ore
da	două etape: **act**, apoi **verify**	fiecare durează minute; așteptarea o face pipeline-ul
da	`toPass({ timeout })` în verify	reîncearcă puțin, dacă jobul întârzie câteva secunde

7. act → fișier → verify

```
ACT  (@act, pașii 1–6)                    VERIFY  (@after-batch, pașii 7–8)
──────────────────────                    ─────────────────────────────────
validează contractul și Ent
salvează ID-urile  ──►  run-data/C-0200.json  ──►  citește run-data/*.json
                        { contractId, entId,       caută DOC_NO în SCHED_ACTIONS
                          docNo }                  așteaptă: toate acțiunile Done
```

Comandă	Ce rulează
`npm run test:act`	tot, fără testele `@after-batch`; scrie `run-data/`
`npm run test:verify`	doar `@after-batch`; fără `run-data/` pică la primul test

8. GitHub Actions

În `.yml`	Înseamnă
`on:`	când pornește
`jobs:`	etapele (act, verify)
`steps:`	pașii unei etape, în ordine
`uses:`	un pas gata făcut (checkout, node)
`run:`	o comandă de terminal
`needs: act`	rulează doar după ce act e verde
artefact	fișiere păstrate după rulare

Declanșator	La job
`push`	la fiecare modificare
`schedule` (cron)	jobul **fix**, de noapte
`workflow_dispatch`	jobul pornit **la cerere**

```
push ──► act ──► verify        (minute)

03:00 ──► verify               (pe ID-urile
buton ──► verify                ultimului act verde)
```